

TRACE 需求说明书

1. 文档说明

1.1 文档目的

本文档面向 TRACE 项目最终交付，用于说明系统建设目标、用户角色、业务流程、功能需求、非功能需求、验收标准和交付边界。本文不是项目早期设想清单，而是以当前代码、演示链路和验收材料为依据整理出的交付版需求说明。

1.2 术语约定

术语	含义
TRACE	Test-generation Reflective Agent for Comparative Evaluation，面向 Python/FastAPI 项目的测试生成、执行、追踪和评测平台
Agent	使用 LLM 和受控工具完成项目分析、测试规划、测试生成、失败反思的自动化执行单元
Project	被测项目，通常是本地 Python/FastAPI 项目
Snapshot	项目快照，用于固定某次运行的代码版本
Runtime Profile	运行配置，描述执行器、pytest 命令、安装命令、环境变量、资源限制、replay policy 等
Strategy Version	不可变策略版本，描述 Direct、Plan-and-Execute、ReAct + Reflection 等生成流程
Test Plan	用户定义的普通测试生成计划，描述目标范围、目标说明和预算
Test Run	一次普通 Agent 测试生成运行
Trace Step	Agent 或系统运行过程中的结构化步骤记录
Eval Dataset	评测数据集，包含任务、seeded bug 和 bug variant
Experiment	一组策略在同一评测数据集上的对比实验
Clean Run	在干净代码上生成并验证测试集的运行
Replay	将冻结后的测试集重放到干净代码或 bug variant 上的执行过程
Seeded Bug	人工设计或确认的已知缺陷
Bug Variant	seeded bug 在代码中的具体变体
MockLLM	内置确定性模型，用于稳定演示和链路验收

2. 项目概述

2.1 项目背景

后端项目通常依赖人工编写单元测试和接口测试。大模型可以生成测试代码，但单纯“生成一段 pytest 文本”不足以进入工程交付：测试必须能真实执行，失败必须能解释，修复过程必须可追踪，不同策略的效果必须能量化比较。

TRACE 的核心问题不是“模型会不会写测试”，而是系统能否证明这些测试：

- 在真实 Python/FastAPI 项目中能被 pytest 收集和执行；
- 失败时能基于结构化错误进行受限反思修复；
- 运行过程、模型输出、工具调用和测试结果可审计；
- 在 seeded bug 数据集上能量化捕获真实缺陷的能力；
- 能通过 Web UI 和 API 形成可复现的交付演示。

2.2 一句话定位

TRACE 是一个面向 Python/FastAPI + pytest 项目的测试生成 Agent 平台，支持项目分析、测试生成、pytest 执行、失败反思、轨迹追踪、报告输出和 seeded bug 策略评测。

2.3 建设目标

本期交付目标如下：

1. 建立完整的测试生成闭环：从项目注册到生成 pytest 测试、执行 pytest、保存结果和展示报告。
2. 建立 Agent 过程可审计能力：记录策略、工具调用、trace step、run event、pytest case result 和 artifacts。
3. 建立受限 Reflection 能力：允许 Agent 基于失败信息修复生成测试，但禁止修改业务代码或削弱核心断言。
4. 建立策略评测能力：使用 seeded bug dataset、clean run、frozen test set replay 和指标聚合比较多种策略。
5. 建立交付演示能力：通过 Docker Compose 启动 PostgreSQL、Redis、Backend API、Worker 和 Frontend，完成 UI 演示和 smoke / benchmark 验证。

3. 用户角色

3.1 后端开发者

后端开发者使用 TRACE 为 Python/FastAPI 项目生成初版 pytest 测试，关注生成测试是否能运行、是否覆盖目标函数或接口、失败原因是否可解释。

主要诉求：

- 注册本地项目；
- 创建快照和测试计划；
- 启动测试生成运行；
- 查看 pytest 结果、失败原因和报告；
- 判断生成测试是否值得采纳。

3.2 测试开发者

测试开发者使用 TRACE 辅助补测、回归测试建设和缺陷捕获评估，关注测试质量、断言有效性、假阳性和缺陷捕获率。

主要诉求：

- 查看用例级 pytest 结果；
- 查看 Agent 生成、反思和修改证据；
- 使用 seeded bug 数据集评估策略；
- 导出 metrics JSON 和报告；
- 区分测试失败、系统失败和真实缺陷捕获。

3.3 Agent 策略研究者

Agent 策略研究者使用 TRACE 比较 Direct、Plan-and-Execute、ReAct + Reflection 等策略，关注同一任务下不同策略的效果、成本、耗时和稳定性。

主要诉求：

- 管理不可变策略版本；
- 在同一 dataset 上创建多策略 experiment；
- 对比 capture rate、false positive、token、工具调用和 replay evidence；
- 追溯每个策略的 clean run 与 replay 细节。

3.4 项目评审者

评审者通过 TRACE 的 UI、Docker Compose 交付路径、演示 runbook 和验收证据判断项目完成度。

主要诉求：

- 一键启动完整产品栈；

- 从 UI 走通 Dataset → Experiment → Start → Metrics / Evidence → Report / Export → Comparison;
- 查看稳定 benchmark 证据;
- 明确系统边界, 尤其是 Docker Compose 与 Docker executor 的区别。

4. 系统范围

4.1 本期交付范围

本期交付范围包含:

- Python/FastAPI + pytest 项目的测试生成和执行;
- 本地项目注册、项目快照、测试计划和普通 test run;
- Direct、Plan-and-Execute、ReAct + Reflection 三类策略版本;
- MockLLM 稳定演示与真实 LLM 可选配置;
- 受控工具调用, 包括项目分析、文件读取/搜索、测试文件写入和 pytest 执行;
- 结构化 trace、run event、pytest case result、artifact 和 run-level report;
- eval dataset、eval task、seeded bug、bug variant 管理;
- mutation discovery dry-run 与 selected mutation confirmation;
- experiment 创建、启动、进度查看、取消和指标查询;
- clean run、frozen test set replay、capture matrix 和 evaluation event;
- runtime profile、preflight、executor status 和 replay policy 展示;
- workspace cleanup / replay cache cleanup 预览、二次确认和结果展示;
- comparison 页面展示 completed experiment 的真实策略指标;
- Docker Compose 产品栈交付。

4.2 非本期范围

以下能力不属于交付承诺:

- 远程 Runner 注册、心跳、long-poll 领任务和跨机器调度;
- 分布式 Runner 集群;
- MinIO/S3 对象存储 artifact;
- 完整 OpenTelemetry 链路追踪;
- 默认启用 Docker executor;
- 完整安全沙箱;
- 任意语言、任意框架测试生成;
- 训练、微调或模型管理平台;
- Jira、TestRail、禅道等测试管理系统替代能力;

- experiment-level Markdown 报告生成服务；
- failed/cancelled experiment 原地 rerun。

5. 总体业务流程

5.1 普通测试生成流程

1. 用户进入项目页，注册或选择一个本地 Python/FastAPI 项目。
2. 系统为项目创建 snapshot，固定本次运行的项目根目录和内容摘要。
3. 用户创建 test plan，填写测试目标、目标范围、策略、预算和是否允许 Reflection。
4. 用户启动 test run。
5. 后端创建运行记录，将策略版本、运行配置和运行时快照固化到 run 级。
6. Worker 异步消费任务，推进 run 状态机。
7. Agent 通过受控工具分析项目结构、路由、函数、依赖和已有测试。
8. Agent 规划测试目标并生成 pytest 测试文件。
9. 执行器运行 pytest，采集 stdout、stderr、退出码、耗时、收集数和用例级结果。
10. 若测试失败且允许 Reflection，Agent 基于结构化失败信息进行一次修复。
11. 系统重跑修复后的测试。
12. 系统保存 trace、run event、pytest result、artifact 和 report。
13. 用户在运行控制台查看概览、轨迹、pytest 结果、产物和报告。

5.2 策略评测流程

1. 用户进入 Dataset 页面，选择或创建 eval dataset。
2. Dataset 包含 eval task、target scope、seeded bug、bug variant 和 ground truth。
3. 用户从 Dataset 进入 Experiment 创建页。
4. 用户选择 dataset、runtime profile、strategy versions、repeat count 和 LLM option。
5. 系统创建 draft experiment。
6. 用户启动 draft experiment。
7. Worker 对每个 task、strategy、repeat 执行 clean run，在干净代码上生成测试并完成 clean validation。
8. 系统冻结最终测试集，形成 generated test set artifact。
9. replay 阶段不再调用 LLM，只将同一批冻结测试集重放到 bug variants。
10. 系统根据捕获口径判断 captured bug、false positive 和 metric status。
11. 用户在 Experiment detail 查看 progress、metrics、capture matrix、clean run、replay evidence、runtime evidence、evaluation event 和 cleanup。
12. 用户可下载 metrics JSON、打开首个 run report，并进入 Comparison 页面查看策略对比。

5.3 Docker Compose 交付流程

1. 用户在仓库根目录执行 `docker compose up --build`。
2. Compose 启动 PostgreSQL、Redis、Backend API、Worker 和 Frontend。
3. API 容器初始化数据库，并 seed strategy 与 demo dataset。
4. 用户打开 `http://127.0.0.1:5186`。
5. 用户切换到 API 数据源，从 UI 走完整演示链路。
6. 用户可运行 `scripts/compose_smoke.ps1` 验证基础链路。
7. 用户可运行 `scripts/compose_benchmark.ps1` 生成三策略 benchmark 证据。

6. 功能需求

6.1 项目管理

系统应支持本地项目注册和查询。

功能要求：

- 创建项目，记录名称、描述、本地路径、语言、框架、来源类型和状态；
- 查询项目列表和项目详情；
- 为项目创建快照；
- 查询项目下的 snapshots、test plans 和 test runs；
- 默认面向 Python/FastAPI 项目；
- 本地路径必须作为受控执行范围的一部分参与路径校验。

6.2 项目快照

系统应使用 snapshot 固定运行输入，避免历史运行随本地项目变化而失去解释。

功能要求：

- snapshot 记录项目 ID、source kind、root path、content hash 和创建时间；
- test run 必须绑定一个 snapshot；
- eval task 必须绑定 project snapshot；
- dataset 可记录相关 snapshot ID 列表；
- 参与运行的 snapshot 不应被原地修改。

6.3 Runtime Profile

系统应支持运行配置管理，用于描述 pytest 执行方式和 replay 策略。

功能要求：

- 创建、查询、更新和归档 runtime profile;
- runtime profile 至少包含 executor、image、working directory、Python 版本、install command、pytest command、env template、resource limits、network policy、artifact policy、cleanup policy 和 replay policy;
- 支持项目级 runtime profile;
- 支持 dataset 级 runtime profile 查询和多项目绑定 manifest;
- 支持 executor status 查询;
- 支持 runtime preflight, 展示 checks、status、replay policy 和 audit-only limitation;
- `local_subprocess` 必须明确标识为本地子进程执行, 不是安全沙箱;
- Docker executor 可作为可选执行器状态展示, 但不作为默认交付承诺。

6.4 策略与版本管理

系统应以不可变策略版本作为普通运行和实验对比的基本单位。

功能要求：

- 创建 strategy;
- 创建和查询 strategy version;
- strategy version 记录 workflow type、模型提供方、模型名称、模型参数、temperature、max tool calls、是否允许 Reflection、prompt version 和 tool schema version;
- strategy version 参与 run 后应锁定或视为不可变;
- test plan 中的默认策略只作为默认值, 最终执行策略必须在 test run 级固化;
- experiment 应支持选择多个 strategy version;
- 已 seed 的核心策略包括 Direct、Plan-and-Execute、ReAct + Reflection。

6.5 Prompt 与工具 Schema 版本

系统应支持 prompt version 和 tool schema version 的版本化记录。

功能要求：

- 创建和查询 prompt version;
- 创建和查询 tool schema version;
- prompt version 保存正文或结构化内容、content hash、source ref 和版本号;
- tool schema version 保存 schema JSON、content hash 和版本号;
- test run 的 strategy snapshot 应包含当时使用的策略、模型、prompt 和工具契约信息, 防止历史结果漂移。

6.6 LLM 选项

系统应提供后端收口的 LLM option。

功能要求：

- 前端通过后端 `GET /api/v1/llm-options` 获取可选模型；
- MockLLM 应始终可用于确定性演示；
- 真实 LLM 只有配置完整且可用时才标记为 selectable；
- 创建 experiment 时的 `llm_override` 必须匹配后端 selectable option；
- LLM config error 响应必须脱敏，不回显 secret、base URL、原始坏配置或本地路径；
- MockLLM 指标和真实 LLM 指标不得混合解释。

6.7 Test Plan

系统应支持普通测试计划创建和查询。

功能要求：

- 创建 test plan；
- 查询 test plan 详情；
- test plan 记录 project、名称、target scope、goal、budget、output options 和默认 strategy version；
- test plan 描述“要测什么”，不保存 Agent 实际拆分出的任务；
- 用户可基于 test plan 启动 test run；
- test run 创建时可覆盖 snapshot、runtime profile、strategy version 和预算。

6.8 Test Run

系统应支持普通 Agent 运行的创建、查询、取消和重试。

功能要求：

- 创建 test run；
- 查询 test run 详情；
- 查询某个 test plan 下的 runs；
- 查询 trace steps、run events、attempts、pytest results、artifacts 和 report；
- 支持取消运行；
- 支持 retry，retry 必须创建新 run，并通过 `retry_of_run_id` 指向原 run；
- test run 必须保存 status、stage、runtime snapshot、strategy snapshot、pytest summary、token、cost、tool call count 和错误信息；

- pytest 有失败用例但系统正常产出报告时，run 可以是 completed，不能简单等同为系统 failed。

6.9 Agent 测试生成

系统应使用受控工具完成测试生成，而不是让 LLM 任意访问仓库或执行 shell。

功能要求：

- Agent 应先分析项目结构、路由、函数、依赖和已有测试；
- Agent 应基于目标范围生成测试计划项；
- Agent 应输出 pytest 测试文件；
- 生成测试应写入受控目录；
- 生成测试应记录目标、假设、覆盖点和风险说明；
- Agent 输出应进行结构化校验；
- 工具调用输入、输出摘要、耗时、token 和错误应写入 trace step；
- Agent 不得直接执行任意 shell。

6.10 Reflection

系统应支持一次受限 Reflection，用于修复生成测试自身的问题。

允许修复的问题包括：

- import 路径错误；
- fixture 缺失；
- 调用签名错误；
- 测试数据构造错误；
- 断言方向明显写反；
- FastAPI TestClient 使用方式不匹配；
- pytest collection 或测试文件结构问题。

禁止行为包括：

- 修改业务源码；
- 删除核心断言；
- 将具体断言改成空洞断言；
- 使用 pytest.skip 掩盖失败；
- 捕获所有异常后强行通过；
- 伪造 pytest 结果；
- 为了通过而接受明显错误的业务行为。

如果不削弱断言就无法通过，系统应保留失败并在报告中标记疑似业务缺陷或测试风险。

6.11 pytest 执行

系统应真实执行 pytest 并保存结构化结果。

功能要求：

- 执行 pytest 时采集退出码、耗时、收集数、通过数、失败数、跳过数、stdout、stderr 和失败详情；
- 保存 pytest case result，至少包含 nodeid、status、duration、failure type、message 和 stdout/stderr 摘要；
- collection error、fixture error、import error、assertion failure、runtime error 应区分记录；
- 支持生成 JUnit XML、日志、生成测试副本和报告等 artifacts；
- 超时、执行器异常、依赖安装失败应转换成结构化错误；
- replay 阶段不得调用 LLM。

6.12 Trace 与事件

系统应完整记录 Agent 和系统运行过程。

功能要求：

- trace step 记录 step index、step type、名称、输入摘要、输出摘要、工具名、payload、token、耗时、状态和错误；
- run event 记录 stage、event type、前后状态、消息和时间；
- 状态更新前应先保存必要的 trace step 或 run event；
- 前端运行控制台应以时间线或列表形式展示 trace；
- trace 用于产品回放，日志用于运维排错，两者不能混成不可查询的长文本。

6.13 Report 与 artifacts

系统应提供 run-level report 和 artifact 查询。

功能要求：

- run-level report 包含 summary、metrics、risk notes、Markdown artifact 和 JSON artifact；
- 前端应展示报告摘要、风险备注、指标、测试质量证据和产物入口；
- artifacts 应记录类型、URI、content hash、大小和 metadata；

- experiment detail 应提供 metrics JSON 下载入口;
- experiment detail 应提供打开首个 run report 的入口;
- 不要求 experiment-level Markdown 报告服务。

6.14 Dataset 管理

系统应支持评测数据集管理。

功能要求:

- 创建和查询 eval dataset;
- 查询 dataset detail;
- dataset detail 展示 task、seeded bug、variant、snapshot、readiness 和 authoring 信息;
- 创建和更新 eval task;
- eval task 记录 project snapshot、target scope、goal 和 expected capabilities;
- 创建和更新 seeded bug;
- seeded bug 记录 bug type、description 和 expected detection;
- 创建和更新 bug variant;
- bug variant 记录 canonical kind、patch artifact、mutated snapshot 和 ground truth;
- dataset readiness 是结构完整性检查, 不是 replay 结果。

6.15 Mutation Discovery

系统应支持 mutation discovery dry-run 和 selected mutation confirmation。

功能要求:

- 对 eval task 发起 mutation discovery dry-run;
- 返回候选 mutation、选中状态、排除原因和目标信息;
- 用户可确认 selected mutation;
- 确认时应由服务端进行 rediscovery 与 probe check;
- 前端结构化 probe 只生成 probe JSON, 信任根仍在后端;
- sampled mutation score 的分母只统计 confirmed 且 probe passed 的 auto mutation variant;
- ordinary variant 不进入 sampled mutation score 分母。

6.16 Experiment 管理

系统应支持 experiment 的创建、查询、启动、取消和进度查看。

功能要求：

- 创建 experiment, 记录 name、dataset、runtime profile、runtime bindings、strategy versions、repeat count、llm override 和 status;
- experiment 初始状态为 draft;
- 只有 draft experiment 可原地 start;
- start 后状态进入 queued/running, 完成后进入 completed, 失败进入 failed, 取消进入 cancelled;
- queued/running/completed 不应重复显示原地 Start;
- failed/cancelled/completed 不支持原地 rerun, 应通过新建 experiment 解决;
- 查询 clean runs;
- 查询 replay runs;
- 查询 metrics;
- 查询 progress;
- 查询 artifact inventory;
- 查询 replay cache;
- 支持 cancel。

6.17 Experiment Metrics

系统应以 seeded bug 捕获率为核心指标, 并明确系统失败与 0 捕获的区别。

核心指标包括：

- capture_rate_mean : 缺陷捕获率均值;
- false_positive_rate : 干净代码假阳性率;
- metric_status : 指标可评价状态;
- cost_per_captured_bug : 每捕获一个 bug 的成本;
- token、工具调用、耗时等工程成本指标;
- replay count、clean run count、cache status 等执行证据。

捕获口径：

- 测试集必须先干净代码上通过或满足可评价条件;
- replay 使用冻结测试集;
- bug variant 上出现与 ground truth 相关的 assertion failure 才算捕获;
- import、collection、fixture、setup、provider、executor、artifact、runtime failure 不算捕获;
- invalid test set 不是 0 捕获;
- provider failure 不是 0 捕获;

- flaky clean replay 不是 0 捕获;
- zero capture 只用于可评价测试集确实没有捕获 bug 的情况;
- false positive 属于 clean baseline 层, 必须和 variant capture 分开。

6.18 Replay Evidence

系统应展示 replay 证据, 证明评测没有污染。

功能要求:

- replay 不规划、不生成、不调用 LLM;
- replay 应记录 `llm_calls=0` ;
- replay 应记录 runtime snapshot、executor metadata、workspace manifest、cache key、cache status、source replay、replay mode、错误码和错误信息;
- 前端应展示 clean replay、variant replay、capturing nodeids、capture rule、错误详情和 prompt boundary;
- replay cache hit/stale/miss 应可见;
- source replay evidence 不应被普通 cleanup 破坏。

6.19 Cleanup

系统应提供 workspace cleanup 和 replay cache cleanup。

功能要求:

- cleanup 必须先 dry-run 预览候选;
- 前端必须二次确认后才能执行删除;
- workspace cleanup 展示 replay ID、status、exists 和 path;
- replay cache cleanup 展示 cache 候选;
- 默认保留失败证据;
- cleanup 只删除 workspace/cache 文件, 不重写历史 metrics、DB 记录或 report;
- 后端必须限制清理范围, 不能删除实验工作区外文件。

6.20 前端 UI

系统应提供完整 Web UI。

页面入口包括:

- 项目;
- 运行控制台;

- 实验;
- 数据集;
- 运行环境;
- 策略对比。

功能要求:

- 支持 API 数据源和静态演示数据源切换;
- 支持中文和英文;
- API 模式展示真实后端数据;
- 静态演示只用于前端预览, 不可作为真实 benchmark 证据;
- Dataset 页面支持列表、详情、readiness、authoring、mutation discovery 和创建 experiment;
- Experiment 页面支持创建、详情、start、progress、metrics、evidence、report/export 和 cleanup;
- Runtime 页面支持 executor status、profile contract、preflight 和 archive;
- Run Console 展示 run overview、trace、report、pytest results 和 artifacts;
- Comparison 页面只展示 completed experiment 的真实 metrics, API 失败时不得回退 demo metrics。

6.21 API 与异步任务

系统后端应提供稳定 API 和异步 worker。

功能要求:

- API 使用 FastAPI 暴露项目、计划、运行、策略、版本、dataset、experiment、runtime、LLM options 等接口;
- Worker 使用 Celery 消费异步运行和评测任务;
- Redis 作为队列和短期状态依赖;
- PostgreSQL 作为主数据存储;
- API 不应执行耗时 Agent 运行;
- Worker 异常、模型失败、pytest 失败、runtime 失败应记录结构化错误;
- API 错误响应应避免泄露敏感配置。

7. 数据需求

详细字段和关系将在数据库设计文档中展开。本文只规定业务数据范围。

系统至少应持久化以下数据：

- projects;
- project snapshots;
- runtime profiles;
- strategies;
- strategy versions;
- prompt versions;
- tool schema versions;
- test plans;
- test runs;
- run plan items;
- run attempts;
- generated test files;
- generated test cases;
- pytest case results;
- trace steps;
- run events;
- run artifacts;
- test reports;
- eval datasets;
- eval tasks;
- seeded bugs;
- bug variants;
- experiments;
- experiment strategy versions;
- experiment clean runs;
- test replays;
- experiment replay runs;
- experiment lifecycle events。

数据设计必须满足：

- run 级可复现;
- 用例级 pytest 结果可查询;
- clean run 与 replay 分离;
- false positive 与 captured bug 分离;
- artifact 不直接塞进主业务表;
- prompt、工具 schema、runtime 和 strategy 能追溯历史版本;

- replay 不依赖可变测试集。

8. 外部接口需求

8.1 Web API

系统应提供 REST 风格 API，供前端和脚本调用。核心接口包括：

- /healthz
- /api/v1/projects
- /api/v1/test-plans
- /api/v1/test-runs
- /api/v1/strategies
- /api/v1/strategy-versions
- /api/v1/prompt-versions
- /api/v1/tool-schema-versions
- /api/v1/eval-datasets
- /api/v1/experiments
- /api/v1/runtime-profiles
- /api/v1/llm-options

API 应使用 Pydantic schema 进行输入输出校验。

8.2 LLM Provider

系统应支持：

- MockLLM;
- OpenAI Responses API;
- OpenAI Chat Completions 兼容接口。

真实 LLM 配置可来自环境变量、`.env` 或本地配置文件。真实 API key 不得提交到 Git。

8.3 pytest 与本地执行器

系统应通过受控执行器运行 pytest。

默认执行器为 `local_subprocess`：

- 本地开发时在宿主机执行；
- Docker Compose 交付时在 worker 容器内执行；
- 不等于安全沙箱；
- 不等于 Docker executor。

8.4 Docker Compose

系统应支持 Docker Compose 一键启动完整产品栈：

- PostgreSQL；
- Redis；
- Backend API；
- Worker；
- Frontend。

Docker Compose 是软件运行交付路径，不是 Docker executor 验收。

9. 非功能需求

9.1 可复现性

系统应保证历史运行可解释、可追溯。

要求：

- test run 固化 snapshot、runtime profile、strategy version、strategy snapshot 和 runtime snapshot；
- experiment 固定 dataset、strategy versions、repeat count、runtime binding 和 LLM option；
- prompt 和 tool schema 具备版本和 content hash；
- replay 使用 frozen test set artifact；
- metrics 可追溯到 clean run、replay、bug variant 和 strategy version。

9.2 可靠性

系统应避免黑盒失败和状态漂移。

要求：

- 运行状态机明确；

- draft experiment 只能 start 一次;
- retry 创建新 run;
- 取消操作幂等;
- 状态更新前先记录事件;
- Worker 失败应保存错误码和错误信息;
- provider/setup/executor/runtime/artifact failure 应与测试捕获失败区分;
- smoke 和 benchmark 脚本可重复运行。

9.3 安全性

系统应控制 Agent 和执行器边界。

要求:

- Agent 只能使用白名单工具;
- 文件读取、搜索和写入必须进行路径校验;
- 生成测试默认写入受控目录;
- 不允许 Agent 修改业务源码;
- 不允许日志和 API 错误泄露 secret;
- Docker Compose 不挂载 Docker socket;
- 不夸大 Docker 或 local subprocess 的安全能力;
- 不在不可信项目上直接执行生成测试。

9.4 可观测性

系统应能解释每次运行和实验。

要求:

- trace step、run event、evaluation event 分层记录;
- run detail 可回放 Agent 轨迹;
- experiment detail 可查看 metrics、evidence、runtime execution JSON 和 cleanup 候选;
- 日志和事件应带 run ID、experiment ID、stage、event type 和错误码;
- metrics JSON 可下载。

9.5 可用性

系统应支持评审和开发者快速上手。

要求:

- Docker Compose 可一键启动;
- 演示路径不依赖手工查数据库;
- 前端导航清晰;
- Dataset 到 Experiment 的创建链路应自动预选 dataset;
- 创建 experiment 前应提示缺失的 dataset、runtime、strategy、LLM option 或 repeat count;
- API 模式和静态演示模式必须可区分;
- 中文 UI 可用于交付演示。

9.6 性能与容量

系统本期以演示和中小型项目验证为目标。

要求:

- demo benchmark 支持 3 个 task、16 个 bug variants、3 个 strategy 和 repeat=3;
- repeat=3 benchmark 应能完成并输出 metrics;
- replay 阶段应支持缓存, 避免重复执行相同 replay;
- 前端 JSON 展示应限制展开深度和数组项数量, 避免大 payload 卡死页面;
- 大体积产物应以 artifact 元数据和文件路径管理, 不直接塞入核心业务表。

9.7 可维护性

系统应保持模块边界清晰。

要求:

- API、service、repository、model、schema、agent、tool、worker 分层;
- 评测逻辑与普通 run 逻辑保持边界;
- 指标口径集中实现, 避免前后端各算一套;
- 测试覆盖核心 API、模型迁移、Agent 工具、评测 harness、cleanup 和 replay scheduler;
- 文档必须与交付边界一致。

10. 验收标准

10.1 基础交付验收

系统应满足:

- `docker compose up --build` 可启动完整产品栈;
- 前端可访问 `http://127.0.0.1:5186` ;
- API 可访问 `http://127.0.0.1:8000` ;
- `/healthz` 返回正常;
- `scripts/compose_smoke.ps1` 通过;
- 可在 API 模式看到 seeded demo dataset;
- 可从 UI 创建、启动并查看 experiment。

10.2 普通 run 验收

系统应满足:

- 可注册或选择项目;
- 可创建 snapshot;
- 可创建 test plan;
- 可启动 test run;
- 可生成 pytest 测试;
- 可真实执行 pytest;
- 可展示 trace、pytest result、artifact 和 report;
- 支持一次受限 Reflection;
- retry 创建新 run, cancel 幂等。

10.3 评测验收

系统应满足:

- demo dataset ID 为 `dataset-demo-v2` ;
- demo benchmark 包含 3 个 task;
- demo benchmark 包含 16 个 bug variants;
- 可创建包含 Direct、Plan-and-Execute、ReAct + Reflection 的 experiment;
- experiment 可从 draft 启动并完成;
- completed experiment 可展示 metrics、capture matrix、clean runs、replay evidence 和 evaluation events;
- replay `llm_calls=0` ;
- invalid/provider/setup/executor/runtime/artifact failure 不被压成 0 捕获;
- false positive 与 captured bug 分开展示;
- 可下载 metrics JSON;
- Comparison 只展示 completed experiment 的真实 metrics。

10.4 已验证参考结果

已验证的 Docker Compose benchmark 证据：

项	结果
Experiment	compose-benchmark-suite-r3-20260622112519
Dataset	dataset-demo-v2
Task 数	3
Bug variant 数	16
Strategy 数	3
Repeat count	3
Clean runs	27
Variant replay records	144
Replay LLM calls	[0]
Metric status	三行 ok

参考指标：

Strategy	capture_rate_mean	false_positive_rate
Direct v 1	0.875	1.0
Plan-and-Execute v 1	1.0	0.0
ReAct + Reflection v 1	1.0	0.0

该结果是 MockLLM 确定性验收证据。真实 LLM 结果应以实际运行的 experiment 为准，不与 MockLLM 指标混讲。

10.5 工程质量验收

系统应满足：

- 后端全量测试通过；
- 前端构建通过；
- 数据库迁移可初始化完整 schema；
- Docker Compose smoke 通过；
- 核心错误响应脱敏；
- cleanup 只影响受控 workspace/cache；

11. 约束与边界

11.1 技术约束

- 主要语言和框架：Python/FastAPI + pytest；
- 后端：FastAPI、SQLAlchemy、Alembic；
- 数据库：PostgreSQL；
- 队列：Redis、Celery；
- 前端：Vue 3、TypeScript、Vite；
- 评测：seeded bug dataset、pytest replay；
- 默认交付：Docker Compose。

11.2 执行边界

- `local_subprocess` 是默认执行方式，但不是沙箱；
- Docker Compose 是产品栈容器化，不是 Docker executor；
- Docker executor 不作为默认验收能力；
- 生成测试本质是 Python 代码，不应在不可信项目上直接运行；
- Docker 不可用应显示为 runtime unavailable 或 preflight warning/failed，不进入捕获率分母。

11.3 评测边界

- replay 不调用 LLM；
- Agent 不得在 bug variant 上重新生成测试；
- Clean baseline 失败属于 false positive；
- 系统失败不是 0 捕获；
- 覆盖率不是核心指标；
- 捕获率分母来自 eval task target scope 下的 bug variants。

12. 主要风险与控制

风险	控制措施
生成测试能跑但断言无意义	保存测试质量证据、断言摘要和人工评分入口，评测以 seeded bug 捕获为核心
Reflection 为了通过削弱测试	Contract Guard 和 Reflection 红线禁止删除断言、skip 和空洞断言
策略版本漂移	run 级固化 strategy snapshot、prompt、tool schema 和 runtime snapshot
系统失败被误算为 0 捕获	使用 metric status 和 evaluation event 区分 invalid/provider/setup/executor/runtime/artifact failure

风险	控制措施
Docker 安全边界被夸大	文档和 UI 明确 local subprocess / Docker Compose / Docker executor 的区别
replay 污染评测	使用 frozen test set replay, 并记录 llm_calls=0
产物清理误删证据	cleanup 先 dry-run, 二次确认, 默认保留失败证据
演示依赖真实 LLM 不稳定	默认 MockLLM 稳定演示, 真实 LLM 作为可选补充

13. 后续扩展需求

后续可考虑：

- 远程 Runner 注册、心跳、long-poll 和租约；
- 分布式 Runner 调度；
- Docker executor 默认化与容器安全加固；
- MinIO/S3 artifact storage；
- 完整 OpenTelemetry；
- 多语言、多框架测试生成；
- experiment-level Markdown 报告服务；
- failed/cancelled experiment clone rerun 合同；
- 更细粒度的人工评分和测试采纳 workflow；
- 更丰富的 mutation operator 与自动变异评估。

这些扩展必须新的设计和验收标准中单独定义，不能反向改变已交付口径。

14. 结论

TRACE 本期交付的是一个可运行、可追踪、可评测的测试生成 Agent 平台。它的核心价值不在于让模型输出测试文本，而在于把测试生成变成一条工程闭环：受控分析项目、生成 pytest、真实执行、结构化失败、受限反思、记录轨迹、生成报告，并用 seeded bug replay 证明不同策略的缺陷捕获能力。

需求边界必须保持朴素和严格：已经交付的能力写进本期范围；尚未完成的 Runner 平台化、对象存储、安全沙箱和完整观测链路写进后续扩展。把边界写清楚，系统才站得住。